A PROJECT REPORT ON

HACKAGON

A Web Platform for Hosting and Joining Online Coding Competitions

INDIVIDUAL CONTRIBUTION REPORT

Member 2: Judge and Admin

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology in Computer Science and Engineering

Submitted by

Harsh Sharma

(Member 2 of the HackAgon team: Hiten Sharma, Harsh Sharma, Hemanth Singh Tanwar, Arjun Suman, Akshat Rathore)

Under the guidance of

[Project Guide]

Department of Computer Science and Engineering

Arya College of Engineering and IT, Jaipur

Session 2026-27

CERTIFICATE

This is to certify that the individual contribution report entitled "HackAgon: Individual Report of Member 2 (Judge and Admin)", submitted by Harsh Sharma, a student of the third year of Bachelor of Technology in Computer Science and Engineering at Arya College of Engineering and IT, Jaipur, is a record of his own work carried out during the session 2026-27 as a member of the HackAgon project team, under my guidance and supervision.

The work embodied in this report has not been submitted elsewhere for the award of any degree or diploma. The report is, in my opinion, worthy of consideration as part of the project submission.

[Project Guide]
Project Guide

[Head of Department]
Head of Department

Date: ____
Place: Jaipur

DECLARATION

I, Harsh Sharma, hereby declare that the individual contribution report entitled "HackAgon: Individual Report of Member 2 (Judge and Admin)" is an original account of the work done by me as Member 2 of the HackAgon project team, under the guidance of the project guide named in the certificate.

I further declare that this report has been prepared from the HackAgon Project Report and the HackAgon Daily Work Log of the team and has not been copied from any other source. Wherever the work of other team members is described, it is attributed to them by name or by member number.

Harsh Sharma

Date: ____
Place: Jaipur

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who helped me complete my part of the HackAgon project. I thank our project guide for constant direction, patient review and helpful suggestions throughout the project. I am grateful to the Head of the Department of Computer Science and Engineering and to the faculty of Arya College of Engineering and IT, Jaipur, for the learning environment and the laboratory facilities.

I thank my teammates Hiten Sharma, Hemanth Singh Tanwar, Arjun Suman and Akshat Rathore. The judge and the admin tools could only be useful because their modules, the submission path, the money and payouts, the proctoring flags and the learning screens, were built in the same weeks and fitted together at the first real tournament. Their reviews of my pull requests and my reviews of theirs taught me as much about collaboration as about code.

I also thank the open-source community. HackAgon stands on Java, Spring Boot, MySQL, TiDB Cloud, Render, GitHub and many small libraries, all of which are free to use and well documented. Finally, I thank my friends and family for their encouragement.

Harsh Sharma

ABSTRACT

HackAgon is a web application for hosting and joining online coding competitions, with solo, duo, squad and duel rooms, paid tournaments with escrow, automatic judging, proctoring and an administrator back office. It was built by five students between 6 July and 6 October 2026. This report describes the contribution of Member 2, Harsh Sharma, who owned two modules: Module 3, the Code Judge and Question Bank, and Module 4, Admin and Moderation.

For Module 3, the report covers the audit of the judge, the code submission and verdict storage, the question API and admin screens, the starter question bank, the filter API, and the hardening of the judge by a concurrency cap, a one-pending-submission rule, reference solutions and a load test with 200 queued submissions. For Module 4, it covers the admin gate, the users list, question and book management, the similarity comparison of submissions, disputes that block payouts, the failed payouts screen with retry, the review tools for cheating and elimination, the final dashboard and the handling of user reports after launch.

A full chapter is devoted to collaboration. Member 2 worked with Member 1 on the submission path, results and organisation approval; with Member 3 on disputes, payouts and security; with Member 4 on cheating flags; and with Member 5 on the question panel, the mistake book, the AI hints and the library. In total Member 2 logged 194 hours over 67 working days. The report separates delivered, simulated and planned features: JavaScript judging is real, while the other languages are simulated.

Keywords: online judge, question bank, moderation, disputes, similarity check, collaboration, Java, Spring Boot, MySQL.

CONTENTS

1. INTRODUCTION 1
1.1 About the HackAgon Project 1
1.2 Purpose of This Report 1
1.3 Member 2 at a Glance 1
1.4 Sources and Method 2
1.5 Organisation of the Report 2
2. ROLE AND RESPONSIBILITIES 2
2.1 The Two Modules 2
2.2 Files, Tables and Endpoints 3
2.3 Decisions Owned by Member 2 3
2.4 What Belonged to Other Members 3
2.5 Effort Summary 4
3. REQUIREMENTS AND DESIGN 5
3.1 Requirements Covered 5
3.2 Non-Functional Requirements That Shaped the Work 5
3.3 Position in the Architecture 6
3.4 Data Design 6
3.5 Design Principles of the Judge 7
4. MODULE 3: CODE JUDGE AND QUESTION BANK 8
4.1 Audit of the Judge (Week 1) 8
4.2 Verdicts and Their Meaning 8
4.3 Limits and Defaults 9
4.4 Code Submission (Week 2) 9
4.5 The Question Create, Read and Update API (Weeks 2 and 3) 10
4.6 Admin Screens for Questions and Test Cases (Week 3) 10
4.7 Starter Questions and Question Growth 10
4.8 The Filter API for the Question Side Panel 11
4.9 Judge Hardening (Week 4) 11
4.10 Judge Load Test and Tuning (Weeks 9 and 10) 11
4.11 Re-checks After the First Tournament 12
5. MODULE 4: ADMIN AND MODERATION 12
5.1 Protecting the Admin Area (3 August) 12
5.2 Users List with Block and Unblock (4 August) 12
5.3 Question Management Screen (5 August) 12
5.4 Book Management Screen (6 August) 12
5.5 Securing the Admin Helper Scripts (7 August) 13
5.6 Similarity Comparison of Submissions (Week 6) 13
5.7 Disputes (Week 7) 13
5.8 Failed Payouts and Retry (Week 8) 14
5.9 Review Tools for Cheating and Eliminated Players (Week 8) 14
5.10 The Final Admin Dashboard (Week 9) 14
5.11 Handling User Reports (Weeks 12 and 13) 15
5.12 The Short Admin Guide 15
6. COLLABORATION WITH OTHER MEMBERS 16
6.1 Overview of Dependencies and Joint Functions 16

6.2 Collaboration with Member 1 (Hiten Sharma)....17
6.3 Collaboration with Member 3 (Hemanth Singh Tanwar)....18
6.4 Collaboration with Member 4 (Arjun Suman)....19
6.5 Collaboration with Member 5 (Akshat Rathore)....20
6.6 Team-Wide Activities....21
6.7 Summary of Collaboration....21
7. TESTING AND VERIFICATION....22
7.1 Test Strategy....22
7.2 Tests Recorded in the Work Log....22
7.3 Results from the Main Report's Smoke Test....22
7.4 Problems Found and Fixed....23
7.5 Known Issues That Remain....23
8. WEEK-BY-WEEK SUMMARY....23
8.1 Weeks 1 and 2: Audit and Judge Core....24
8.2 Weeks 3 and 4: Question Bank and Judge Hardening....24
8.3 Weeks 5 and 6: Admin Back Office and Similarity....24
8.4 Weeks 7 and 8: Disputes, Payouts and Review Tools....25
8.5 Weeks 9 and 10: Dashboard, Questions and Load....25
8.6 Weeks 11 and 12: Bug Bash and Launch....25
8.7 Final Period: Closing the Loop....25
9. CHALLENGES, DECISIONS AND LIMITATIONS....25
9.1 Challenges and How They Were Met....25
9.2 Delivered, Simulated and Planned....26
9.3 Notes on the Sources....26
9.4 Limitations....27
10. CONCLUSION AND FUTURE SCOPE....27
10.1 Conclusion....27
10.2 Lessons Learnt....27
10.3 Future Scope....28
REFERENCES....28
APPENDIX A: COMPLETE DAILY WORK LOG OF MEMBER 2....29
APPENDIX B: DATED JOINT WORK WITH OTHER MEMBERS....31

LIST OF FIGURES

Figure 2.1: Hours logged by Member 2 per area of work....4
Figure 2.2: Timeline of Member 2's work across the project weeks....5
Figure 3.1: System architecture (from the main project report)....6
Figure 3.2: Match, judging and proctoring flow (from the main project report)....6
Figure 4.1: Admin back office: question bank tab (from the main project report)....10
Figure 5.1: Where the judge, similarity check and flag list fit in the submission path....13
Figure 5.2: Dispute, payout hold and retry across Members 1, 2 and 3....14
Figure 6.1: Collaboration map of Member 2 with the other four members....17

LIST OF TABLES

Table 1.1: Summary of Member 2....1
Table 2.1: Files, tables and endpoints of Member 2....3
Table 2.2: Hours by area of work....4
Table 3.1: Requirements and Member 2's contribution....5
Table 3.2: Tables used by Member 2....6
Table 4.1: Week 1 audit activities of Member 2....8
Table 4.2: How a run is turned into a stored verdict....8
Table 4.3: Limits relevant to the judge....9
Table 4.4: Growth of the question bank....11
Table 5.1: Admin tools built by Member 2....14
Table 6.1: Dependencies of Member 2's modules (from the main report)....16
Table 6.2: Matrix of joint functions....16
Table 7.1: Tests carried out by Member 2....22
Table 7.2: Smoke test cases that concern Member 2....22
Table 8.1: Weekly summary of Member 2's work....23
Table 9.1: Challenges and responses....25
Table 9.2: Status of Member 2's features....26
Table A.1: Daily work log of Member 2....29
Table B.1: Dated joint work....31

1. INTRODUCTION

1.1 About the HackAgon Project

HackAgon is a web platform for hosting and joining online coding competitions. It lets a college club, a class or a group of friends build a contest in a few minutes, pick questions from a shared bank, invite players and, when they want stakes, run a paid tournament in which the prize pool is funded in advance and the platform pays the prizes by a published split after a flat 1% fee. The system is built with Java and Spring Boot, a MySQL-compatible database on TiDB Cloud, and fifteen plain HTML pages. It exposes 76 REST endpoints across eight controller classes and uses a twenty-table schema.

The project was carried out by a team of five students of Arya College of Engineering and IT, Jaipur, between 6 July 2026 and 6 October 2026. The work was divided into ten modules, and every member owned two of them. The team plan numbers the members from 1 to 5. This report is about **Member 2, Harsh Sharma**, whose area in the plan is **Judge and Admin**.

1.2 Purpose of This Report

The main project report describes the whole system and names the owner of each module in one short paragraph. It cannot describe every contribution in detail. The purpose of this individual report is to record, in one place, everything that Member 2 did during the project: what was built, in which week, how it was tested, and how it connected to the work of the other four members.

Special attention is given to collaboration. HackAgon is a tightly connected system. A submission written by one member is judged by code written by another, a dispute raised in one module blocks a payout handled by a third, and a cheating flag produced by a fourth member is reviewed in an administrator screen. Wherever Member 2 worked together with another member on a shared function, this report shows the shared function, the interface between the two sides and the dates on which each side did its part.

1.3 Member 2 at a Glance

Table 1.1: Summary of Member 2

Item	Details
Name	Harsh Sharma (Member 2)
Role in the team plan	Judge and Admin
Modules owned	Module 3: Code Judge and Question Bank Module 4: Admin and Moderation
Main files	`JudgeService.java, PlayController.java (shared), ContentController.java, AdminController.java, admin.html, the make-admin script`
Main tables	questions, submissions, mistakes, books, disputes, users
Project period	6 July 2026 to 6 October 2026
Work logged	67 weekdays, all marked Completed; 194 hours (2 to 4 hours a day, 14.5 in each full week)

Item	Details
Collaborated with	Member 1 (Hiten Sharma), Member 3 (Hemanth Singh Tanwar), Member 4 (Arjun Suman), Member 5 (Akshat Rathore)

1.4 Sources and Method

This report was prepared from two documents: the HackAgon Project Report and the HackAgon Daily Work Log. The project report supplies the design, the module descriptions, the database, the test results and the honest status of each feature. The daily work log supplies the day-by-day record of what each member did, including the 67 entries for Member 2 and the entries of the other four members that touch Member 2's work. Dates in this report follow the day-month-year format of the log.

Where the two documents differ in detail, this report follows a simple rule. The project report is the authority on what the delivered system contains and which parts are real or simulated. The work log is the authority on what activities Member 2 carried out and when. A few activities in the log, such as the similarity check between submissions, the cap on concurrent judge runs and the payout retry button, are not described separately in the project report; they are reported here as recorded in the log. Section 9.3 lists these points openly.

1.5 Organisation of the Report

Chapter 2 explains the role, the files and the effort of Member 2. Chapter 3 lists the requirements and the design that the two modules follow. Chapter 4 describes Module 3, the code judge and the question bank, and Chapter 5 describes Module 4, the administrator back office and the moderation tools. Chapter 6 is devoted to collaboration with the other members. Chapter 7 reports testing, Chapter 8 gives a week-by-week account of the work, and Chapter 9 discusses challenges, decisions and limitations. Chapter 10 concludes with lessons learnt and future scope. Appendix A reproduces the complete daily log of Member 2 and Appendix B lists the dated joint work with the other members.

2. ROLE AND RESPONSIBILITIES

2.1 The Two Modules

In the team plan every member owns one larger and one smaller module so that the load is shared evenly. Member 2 owns the two modules that decide whether a contest is judged fairly and whether it can be managed after it ends.

Module 3: Code Judge and Question Bank

This module evaluates submissions and serves questions. A submission is run against hidden test cases, the verdict is stored, the score is awarded, and a wrong answer is saved as a mistake for later study. The module also holds the question bank from which hosts choose questions, and the practice endpoint that lets a player retry a question outside any room. All judging sits behind one module, `JudgeService.java`, so that a hosted judge service such as Judge0 can replace the in-process judge in future without touching the routes.

Module 4: Admin and Moderation

This module is the back office of the platform. Administrators use it to manage the question bank and the library, review accounts, resolve disputes and watch the money totals. A dispute blocks the payout of its room until an administrator decides. The module also contains the tools that review cheating flags and eliminated players. The administrator gate checks the admin flag on the server for every call, not only in the page.

2.2 Files, Tables and Endpoints

Table 2.1: Files, tables and endpoints of Member 2

File	Endpoints	Responsibility
JudgeService.java	n/a	Runs a submission against test cases and returns a verdict; the only place where code is executed
ContentController.java	7	Bootstrap data, questions, books and taxonomies (topics, languages) for the browser, with hidden cases removed
PlayController.java	6	Submit, practice, mistakes and notifications. Shared file: the submission endpoint was written by Member 1 and calls the judge
AdminController.java	14	Questions, books, users, disputes and room inspection
admin.html	n/a	Back-office page with tabs for the question bank, library licences, accounts, disputes and money
The make-admin script	n/a	Script that promotes an account to administrator; protected in deployment

Together, `ContentController.java and AdminController.java account for 21 of the 76 endpoints of the system, and PlayController.java` adds six more that are shared with Member 1. The tables most used by Member 2 are questions, submissions, mistakes, books, disputes and users.

2.3 Decisions Owned by Member 2

- **One judge module.** All execution and verdict logic lives in `JudgeService.java`. The routes only call it. This keeps the replacement of the judge a local change.
- **Hidden cases never leave the server.** The bootstrap endpoint strips hidden test cases and sends only visible examples to the browser.
- **Honest labelling of simulated verdicts.** Where a language is not really executed, the response carries a simulated flag so that the interface and any audit can tell the two kinds of verdict apart.
- **Server-side admin gate.** Every administrator route checks the admin flag on the server; hiding a button in the page is never treated as protection.
- **Disputes as a hard stop.** An open dispute is a record in the database that the money module reads; it blocks settlement until an administrator decides.

2.4 What Belonged to Other Members

Clear boundaries kept the team from overlapping. Member 2 did not own the room lifecycle or the submission endpoint (Member 1), the payment, settlement and payout logic (Member 3), the proctoring events, snapshots and team scoring (Member 4), or the look of the screens and the mistake book (Member 5). In each of these areas, Member 2 supplied or consumed a well-defined piece, and those pieces are described in Chapter 6.

2.5 Effort Summary

Member 2 logged work on all 67 weekdays between 6 July and 6 October 2026, a total of 194 hours. Daily entries range from 2 to 4 hours. Each full week contains 14.5 hours, and the last three days of the project, which fall in the final partial week, add 5.5 hours to the final period. The log was grouped into seven areas of work to show where the effort went.

Table 2.2: Hours by area of work

Area of work	Days	Hours	Share
Judge engine and submissions	13	38.5	19.8%
Testing, load and bug fixing	12	34.0	17.5%
Admin back office	10	29.5	15.2%
Integrity: similarity and review tools	8	25.5	13.1%
Disputes and payouts	9	25.5	13.1%
Question bank	8	23.0	11.9%
Support, review and documentation	7	18.0	9.3%
Total	67	194.0	100%

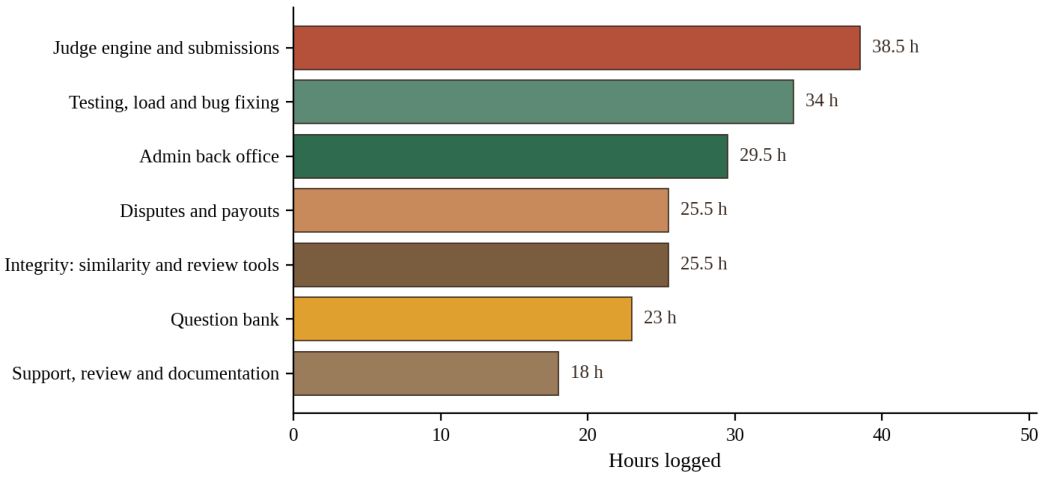


Figure 2.1: Hours logged by Member 2 per area of work

The largest share went to the judge, which is the technical core of Member 2's role. Testing and bug fixing came second, because the judge and the admin tools are only useful if they behave correctly under bad input and under load. Figure 2.2 shows when each area of work took place across the thirteen weeks.

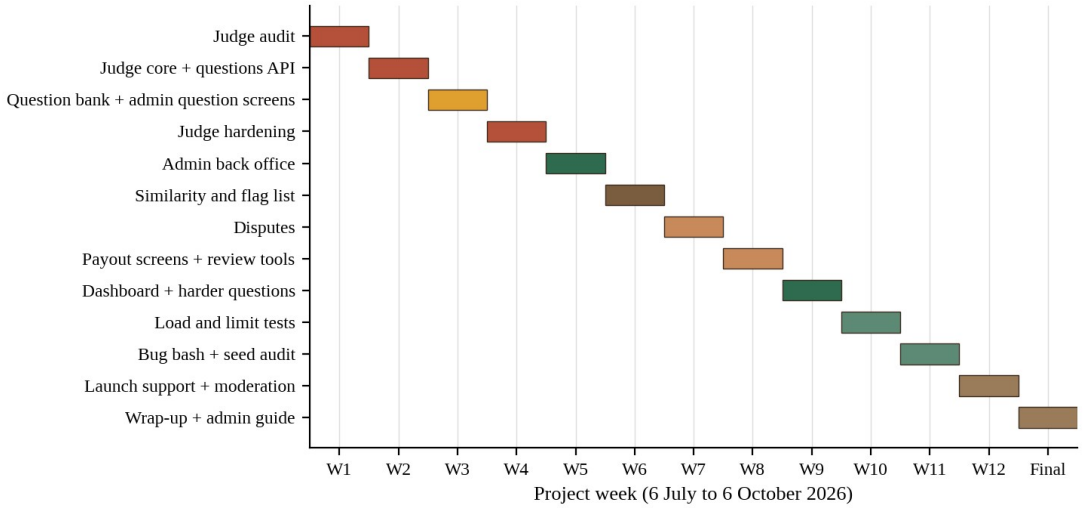


Figure 2.2: Timeline of Member 2's work across the project weeks

3. REQUIREMENTS AND DESIGN

3.1 Requirements Covered

The project requirements are listed as functional requirements FR1 to FR8 in the main report. Member 2 is the main owner of FR4 and FR8 and a contributor to two others.

Table 3.1: Requirements and Member 2's contribution

ID	Requirement (short form)	Contribution of Member 2
FR4	Submit code, judge it against hidden test cases, award points plus a time bonus, record failures as mistakes	Judge module, submission handling with language and question id, verdict storage, known good and bad solutions, limits
FR8	Administrator tools for the question bank, library review, users, disputes and money totals	Admin API and screens: questions, test cases, books, users, disputes, failed payouts, dashboard
FR5	Camera gate, snapshots, tab-lock, host unlock	Review tools for flagged cheating and eliminated players; similarity flags next to proctoring flags (with Member 4)
FR6	Finalise a room, settle the prize pool, ledger	Dispute that blocks settlement; failed payout screen and retry (with Members 1 and 3)

3.2 Non-Functional Requirements That Shaped the Work

- **Security.** Hidden test cases never reach the browser, administrator access is verified on the server, and helper scripts that can create administrators are protected in deployment.
- **Reliability under load.** The judge must stay responsive when many submissions arrive together, so concurrent runs are limited and one pending submission per player per question is allowed.
- **Fairness.** Verdicts are deterministic for the same code and cases, and submitted code is kept so that similar submissions in a room can be compared.

- **Maintainability.** Judging, content delivery and administration are separate route files and one library module, as required by the main report.

3.3 Position in the Architecture

HackAgon is a three-tier application served from a single origin. The browser calls the JSON API under the path /api, and the server talks to the database through a pool of connections and to two external services, Gmail SMTP and the Groq API. Member 2's modules sit entirely inside the server tier. The judge is called from the submission endpoint, the content routes feed the browser, and the admin routes serve the back-office page.

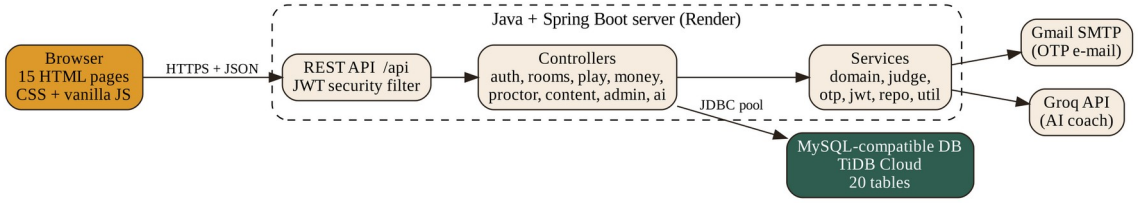


Figure 3.1: System architecture (from the main project report)

The match flow in the main report shows where the judge fits. After the camera gate and the lock check, each submission is judged and either scores or becomes a mistake. Member 2's part of that flow is the judging step itself and everything that reads its output afterwards.

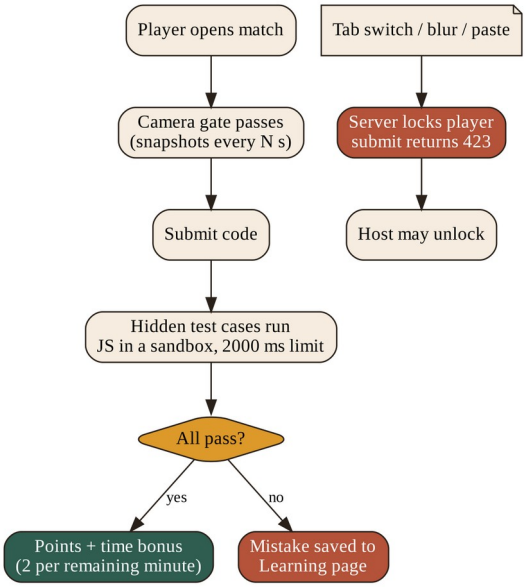


Figure 3.2: Match, judging and proctoring flow (from the main project report)

3.4 Data Design

Member 2's modules read and write six of the twenty tables. The main report describes their columns; the table below restates only the ones that matter for the work in this report.

Table 3.2: Tables used by Member 2

Table	Important columns	Used for
questions	id, title, topics, difficulty, points, visible examples, hidden test cases, source	Question bank, admin edit and retire, filters

Table	Important columns	Used for
submissions	id, room_id, user_id, question_id, lang, code, passed, total, ok, time_ms, simulated, at	Verdicts, scoring, similarity checks
mistakes	user and question, last code, failing case	Wrong answers handed to the mistake book (Member 5)
books	title, licence note, approval status	Library review in the admin screen
disputes	room, raised by, reason, status	Player disputes that block payout
users	id, username, email, is_admin, org (JSON), stats	Users list, block and unblock, organisation approval

The `simulated` column of `submissions` is an important design choice. It records whether a verdict came from real execution, so that the report, the interface and any audit can tell the two apart. The schema file is owned by Member 3, who fixed the V1 tables on 13 July; changes to shared files such as the schema were requested through pull requests, as the main report describes.

3.5 Design Principles of the Judge

1. **Separation.** The routes never run code. They pass the language, code and question id to the judge and receive a verdict.
2. **Isolation of secrets.** The judge reads hidden cases from the database; the browser is never given them.
3. **Bounded execution.** Every run has a time limit and memory limit that were confirmed in the first week and are enforced for all runs.
4. **Replaceability.** The input and output of the judge are small and fixed, so an external judge service can be plugged in without changes elsewhere.
5. **Truthful output.** A verdict states whether it is real or simulated.

4. MODULE 3: CODE JUDGE AND QUESTION BANK

This chapter describes the first of Member 2's two modules. It follows the order in which the work was done: first an audit of the existing judge code, then the submission and question features, then the hardening that made the judge safe to use with many players.

4.1 Audit of the Judge (Week 1)

The project began with a week of audit and decisions. Member 2 spent the first five days on the judge. On 6 July the existing `JudgeService.java` was tested with a sample Java program, and on 7 July with Python and C++ programs. On 8 July the time and memory limits were confirmed, which took four hours, the longest single day of the week. On 9 July Member 2 noted how each verdict maps to a result, and on 10 July wrote down which judge service is used and what its limits are.

This audit had two purposes. The first was to learn exactly what the judge could and could not do before building features on top of it. The second was to give the team an honest basis for its decisions. The main report records the outcome: JavaScript submissions are really executed inside a restricted sandbox context with a two-second wall-clock timeout, while Java, Python, C++ and C are judged by a deterministic simulation and are marked as simulated. The module comment in the source states plainly that the in-process sandbox is not a hardened security boundary, and that a hosted service such as Judge0 should replace it in a product build.

Table 4.1: Week 1 audit activities of Member 2

Date	Activity	Result
06-07-2026	Tested judge with a sample Java program	Behaviour of the judge on a Java submission known
07-07-2026	Tested Python and C++ programs	Behaviour on the remaining languages known
08-07-2026	Confirmed time and memory limits	Limits fixed for all later work
09-07-2026	Noted how each verdict maps	Verdict-to-result table written down
10-07-2026	Wrote down which judge service is used and its limits	Decision record for the team

4.2 Verdicts and Their Meaning

A judge must turn the result of running code into a small, fixed set of outcomes. The table below shows the outcomes the judge distinguishes and how they appear in the stored record. The columns follow the `submissions` table of the main report: `ok` tells whether all cases passed, `passed` and `total` count the cases, `time_ms` stores the run time, and `simulated` marks verdicts that were not produced by real execution.

Table 4.2: How a run is turned into a stored verdict

Situation	How it is recorded	Effect
All hidden cases give the expected output	ok = true, passed = total	Points plus time bonus awarded

Situation	How it is recorded	Effect
Some cases give wrong output	ok = false, passed < total	No points; saved as a mistake
Run exceeds the time limit	ok = false; run stopped at the limit	No points; saved as a mistake
Code fails to run or throws an error	ok = false, passed = 0	No points; saved as a mistake
Language is not really executed	Same fields plus simulated = true	Shown to the user as a simulated verdict

The mapping in the middle column is conceptual: it describes the meaning of the stored fields, which is what the work log refers to as noting how each verdict maps.

4.3 Limits and Defaults

Table 4.3: Limits relevant to the judge

Item	Value or rule
JavaScript run time	2000 ms wall-clock per run, then the run is stopped
Hidden test cases	Run for every submission; never sent to the browser
Languages offered	Java, Python, C++, C and JavaScript
Pending submissions	One per player per question
Concurrent judge runs	Capped (added on 27-07-2026)
Scoring	Question points plus 2 points per whole minute remaining in the room

4.4 Code Submission (Week 2)

In the second week the audit turned into features. On 13 July Member 2 built the code submit function, which accepts a language and a question id together with the code. On 14 July the judge was made to run every test case of a question rather than stopping at the first. On 15 July the verdict, the time and the memory of each run were stored. These three days, which took 9.5 hours in all, gave the platform its first complete path from a piece of code to a stored result.

A submission names the room, the question, the language and the code. The judge wraps the user's code, runs each hidden case, compares the result with the expected value and returns the number of cases passed and failed. The scoring logic that follows is simple and transparent, which is why it was easy to test. The main report shows the time bonus as follows.

```
const timeBonus = Math.max(0,
  Math.round((room.durationMin * 60 - elapsedSecs) / 60) * 2);
const gained = q.points + timeBonus;
```

In the smoke test of the main report, a correct JavaScript solution passed four of four hidden cases and scored 140 points, while a wrong answer failed and was booked in the mistake book. A non-JavaScript verdict was correctly flagged as simulated.

4.5 The Question Create, Read and Update API (Weeks 2 and 3)

The question bank is the supply side of every contest. On 16 July Member 2 built the question create and read API, and on 17 July the update API, which was tested the same day. The read side is used by the creation forms and the duel picker; the create and update side is used by administrators. The bootstrap endpoint in `ContentController.java` returns questions to the browser with the hidden test cases removed, so a player can read the problem and the visible examples but can never see the cases that decide the verdict.

Questions can be created, edited and retired. Retiring is used instead of deleting, so that a question can leave the pick lists while old submissions and mistakes that refer to it stay in the database.

4.6 Admin Screens for Questions and Test Cases (Week 3)

In the third week the API was given a face. On 20 July Member 2 built the admin screen to add questions, and on 21 July the admin screen to add test cases. Each question therefore has a title, topics, a difficulty level, points, visible examples and a list of hidden cases. The back-office page shows the question bank as a table with search, topic, difficulty and source filters; each row shows the question, its topics, level, points, number of cases and source (seeded or added by an administrator), with an edit and a retire action.

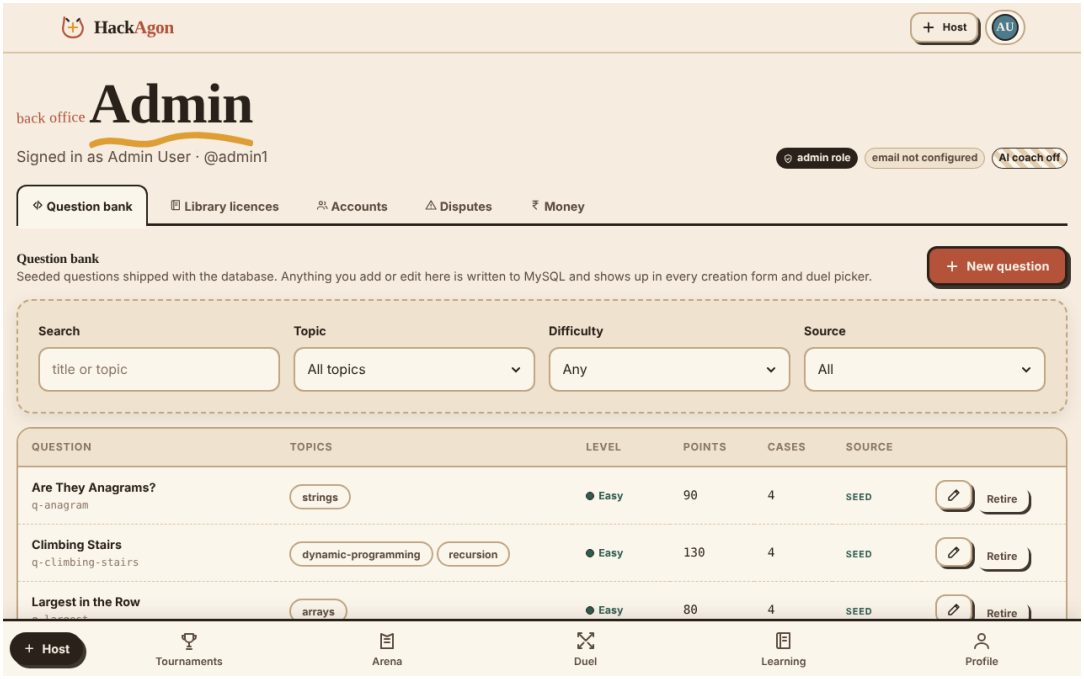


Figure 4.1: Admin back office: question bank tab (from the main project report)

4.7 Starter Questions and Question Growth

A platform with no questions is empty, so Member 2 loaded starter content. On 22 July the starter questions for arrays and strings were loaded, and on 23 July those for linked lists, stacks, queues and trees. The seed script of the project fills the database with nineteen questions that cover arrays, strings, linked lists, stacks, queues, trees, graphs, sorting, recursion and dynamic programming, and the admin screen shows them with the source label SEED. Later, on 3 September, medium questions were added for every topic and, on 4 September, hard questions

for every topic. A final batch was added to the question bank on 30 September, after the first real tournament had shown which topics players asked for.

Table 4.4: Growth of the question bank

Date	Work	Purpose
22-07-2026	Starter questions: arrays and strings	First content for the Arena
23-07-2026	Starter questions: linked lists, stacks, queues, trees	Cover the data-structure topics
03-09-2026	Medium questions for every topic	Give hosts a range of difficulty
04-09-2026	Hard questions for every topic	Support serious tournaments
30-09-2026	More questions added to the bank	Respond to the first event

The seed files contain only reference data. On 18 September Member 2 confirmed this, which matches the clean-up of fake data from the seed files that Member 3 had done on 9 July.

4.8 The Filter API for the Question Side Panel

The create-room page has a side panel from which the host picks questions. On 24 July Member 2 built the filter API behind it, so that questions can be narrowed by topic and difficulty. The panel itself was designed by Member 5 on 20 and 21 July, so this feature is one of the clearest joint functions of the project; its two halves are described in Section 6.5.

4.9 Judge Hardening (Week 4)

The fourth week made the judge reliable enough for the first playable match, which was Milestone 1 on 2 August. Five pieces of work were done.

1. **Concurrency cap (27 July).** A limit on the number of judge runs that may happen at the same time stops a burst of submissions from exhausting the server.
2. **One pending submission per player per question (28 July).** A player cannot queue several runs of the same question, which prevents both accidental double clicks and deliberate flooding.
3. **Known good solutions (29 July).** For every language Member 2 wrote solutions that must pass, a four-hour day. They act as a regression test for the judge.
4. **Known bad solutions (30 July).** Matching solutions that must fail were written and the verdicts were checked, so that the judge is proven to reject as well as to accept.
5. **Stored code for similarity (31 July).** Submitted code is kept in the submissions table so that submissions can be compared later.

Together, the known good and known bad solutions form a small but effective test suite. After any change to the judge, running them shows within minutes whether a verdict has changed.

4.10 Judge Load Test and Tuning (Weeks 9 and 10)

The final hardening was a measured load test. On 7 September Member 2 prepared a script that sends 200 queued submissions to the judge. On 8 September the load test was run and the results were noted, on 9 September the concurrency setting was tuned using those results, on 10 September huge inputs and infinite loops were tested, and on 11 September Member 2

confirmed that all limits are enforced. The load test was run in the same week as Member 3's simulation of 200 and then 500 players polling one room, so the team could judge the full system and not only its parts.

The infinite-loop test is the most important of these. A program that never ends must be stopped at the time limit and reported as a failed run, and it must not delay other players. Because the earlier steps had added the time limit, the concurrency cap and the one-pending rule, the test confirmed that the three protections work together.

4.11 Re-checks After the First Tournament

After the first real tournament on 24 September, Member 2 re-ran the judge checks for every language on 1 October. The re-run used the known good and known bad solutions of Week 4 and confirmed that no change made during the launch weeks had altered a verdict.

5. MODULE 4: ADMIN AND MODERATION

The second module is the back office. Administrators need to manage content, watch for misuse and settle disagreements. Member 2 built these tools in the second half of the project, after the judge was stable.

5.1 Protecting the Admin Area (3 August)

The first step was access control. On 3 August Member 2 added admin login protection. Every administrator route verifies the signed session token and then checks the admin flag of the user on the server. The main report states the rule clearly: the gate checks the flag for every call, not only in the page. Addresses listed in the `ADMIN_EMAILS` setting become administrators at sign-up. The session token itself is created by the accounts module of Member 1, so the admin gate depends on the work of Member 1 (Section 6.2).

5.2 Users List with Block and Unblock (4 August)

The accounts tab lists the users of the platform and lets an administrator block or unblock an account. Blocking is the simplest effective moderation tool: a user who has been found cheating or abusing the platform can be stopped without deleting the evidence of what happened. The list also shows the organisation information that Member 1 added in the last week of August, which is described in Section 6.2.

5.3 Question Management Screen (5 August)

Building on the screens of Week 3, a question management screen was added on 5 August. It collects the actions that were scattered over several screens: searching, filtering, editing, adding test cases and retiring. This is the table shown in Figure 4.1. The screen was used throughout the project to correct questions that turned out to be unclear.

5.4 Book Management Screen (6 August)

The library of free books is part of the learning feature. Each book carries a licence note and an approval status, so that only books that can legally be shared are shown to players. On 6 August Member 2 built the book management screen on which an administrator reviews a book, checks

its licence and approves or rejects it. Member 5 built the page that displays the approved books to players on 13 and 14 August; the join between the two is described in Section 6.5.

5.5 Securing the Admin Helper Scripts (7 August)

The repository contains scripts that are useful in development but dangerous in production, because they can turn an ordinary account into an administrator. On 7 August Member 2 protected `the make-admin script and removed the fix-admin script from the deployment. Member 3 repeated the check from the deployment side on 4 September, when the make-admin script and the fix-admin script` were locked in production. The two entries show the same risk handled from two sides.

5.6 Similarity Comparison of Submissions (Week 6)

Week 6 added an integrity tool that is not described in detail in the main report but is clear in the work log. Because the submitted code is stored, the platform can compare submissions inside one room. On 10 August Member 2 wrote the similarity comparison, on 11 August tested it on sample submissions, on 12 August produced the flag list for the host dashboard, on 13 August showed the flag list in the admin screen, and on 14 August fixed bugs and tuned the threshold.

A similarity score is a hint, not a proof. Two honest players can write the same short solution to a simple question. For this reason the tool produces a list for human review, with a tuned threshold, and never eliminates anyone by itself. The review tools of Week 8 give the administrator and host the means to decide.

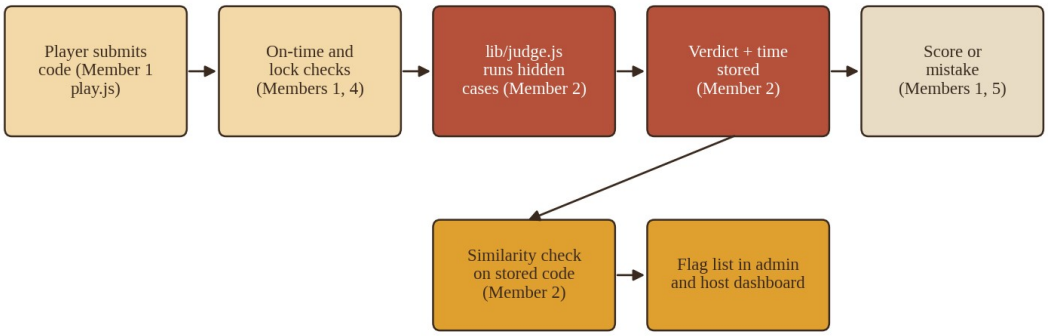


Figure 5.1: Where the judge, similarity check and flag list fit in the submission path

5.7 Disputes (Week 7)

A paid tournament needs a way to say that a result is wrong. In Week 7 Member 2 built the dispute feature in five steps. On 17 August the player dispute form was built. On 18 August the dispute was made to block the payout, which means that the money module will not settle a room while a dispute is open. On 19 August the admin dispute review screen was built, on 20 August the decide and close actions were added, and on 21 August the whole dispute flow was tested.

The result is a clear sequence. The host confirms the result, or it is confirmed automatically after 24 hours without a dispute (the rule built by Member 1). A player may raise a dispute before that point, the dispute holds the payout, an administrator reviews it and decides, and only

then can settlement continue. Because the dispute is a record with a status, the history of every disagreement stays available for audit.

5.8 Failed Payouts and Retry (Week 8)

Payouts can fail for reasons outside the platform, such as an invalid account. On 24 August Member 2 built a screen listing failed payouts, and on 25 August added a retry button that asks the payout to run again. The screen only presents and triggers; the payout logic itself belongs to Member 3. In the main report the payment gateway is simulated in this build, so the failed-payout list shows the records that the money module marks as failed, and a real gateway would produce more realistic failures in future.

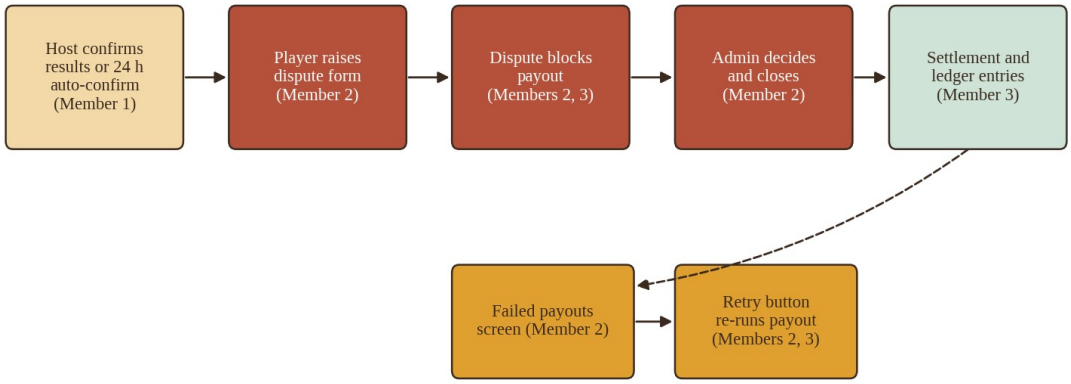


Figure 5.2: Dispute, payout hold and retry across Members 1, 2 and 3

5.9 Review Tools for Cheating and Eliminated Players (Week 8)

On 26 August Member 2 built the review tool for flagged cheating, and on 27 August the review tool for eliminated players. The first lets an administrator see the flags raised in a room, whether they come from proctoring violations or from the similarity list, and decide what to do. The second shows the players that a host has eliminated, with the reason that was given, so that an administrator can check that eliminations were fair. On 28 August Member 2 tested all of the admin tools together.

5.10 The Final Admin Dashboard (Week 9)

In Week 9 the separate screens were brought together. On 31 August the final admin dashboard layout was built, on 1 September the disputes and payout failures panels were added, and on 2 September the flagged cheating panel was added. The dashboard gives an administrator one view of everything that needs attention, instead of a search through several tabs. The tabs of the back-office page are the question bank, library licences, accounts, disputes and money.

Table 5.1: Admin tools built by Member 2

Tool	Date	Purpose
Admin login protection	03-08-2026	Server-side admin gate
Users list, block and unblock	04-08-2026	Account moderation

Tool	Date	Purpose
Question management	05-08-2026	Search, edit, retire questions
Book management	06-08-2026	Licence review of library books
Flag list	12 to 13-08-2026	Similarity flags for host and admin
Dispute review	17 to 20-08-2026	Raise, hold, decide and close disputes
Failed payouts and retry	24 to 25-08-2026	Handle payouts that did not complete
Cheating and elimination review	26 to 27-08-2026	Review flagged and eliminated players
Final dashboard	31-08 to 02-09-2026	One view of disputes, payouts and flags

5.11 Handling User Reports (Weeks 12 and 13)

When the platform went live the team needed a way to handle reports from real users. On 21 September Member 2 set up the handling of user reports, and on 22 September reviewed the first reports. On 23 September flagged cheaters were reviewed, and on 24 September the disputes from the first event were reviewed. After the retro on 25 September the work continued: on 28 September the remaining user reports and flagged cheating were reviewed, and on 29 September the open disputes were closed. This was the first time the back office was used with real data, and every tool built in the previous weeks was used.

5.12 The Short Admin Guide

On 5 October Member 2 wrote a short admin guide that explains the back office to a new administrator: how to add and edit a question, how to check a book licence, how to review a flag, how to decide a dispute and what to do when a payout fails. The guide matches the tabs of the final dashboard and is the last deliverable of the module, together with the final review on 6 October.

6. COLLABORATION WITH OTHER MEMBERS

No module of HackAgon works alone. The main report states that the modules are loosely coupled, but that a few depend on others, and it lists the member pairs who agree each interface. This chapter looks at the same facts from the side of Member 2. For each other member it shows the functions on which the two worked together, what each side built, and the dates recorded in the daily logs. The dates matter, because they show that the two halves of every shared function were built in the same weeks and tested together.

6.1 Overview of Dependencies and Joint Functions

The main report lists the following dependencies for the modules of Member 2. In each case two members agree the interface before either side builds against it.

Table 6.1: Dependencies of Member 2's modules (from the main report)

Module	Depends on	Interface agreed by
Code judge and question bank	Rooms (submission context)	Harsh Sharma and Hiten Sharma
Admin and moderation	Accounts, money, question bank	Harsh Sharma and Hemanth Singh Tanwar
Duels and teams (Member 4)	Rooms, judge	Arjun Suman and Harsh Sharma
Learning, achievements and AI (Member 5)	Judge (mistakes), rooms (results)	Akshat Rathore and Harsh Sharma

From the logs, nine joint functions can be identified. They are summarised in the matrix below and then described member by member.

Table 6.2: Matrix of joint functions

Joint function	With	Member 2 built	Other member built
Submission and verdict	Member 1	Judge, run of all cases, stored verdict	Submission endpoint, on-time check, judge call
Elimination review	Members 1, 4	Review tool for eliminated players	Eliminate logic (M1), eliminate button (M4)
Disputes, results and payout	Members 1, 3	Dispute form, block, review, close	Confirm and 24 h auto-confirm (M1), settlement (M3)
Failed payouts	Member 3	Failed payouts screen, retry button	Payout job, Razorpay account check
Security of admin scripts	Member 3	Protected the make-admin script, removed the fix-admin script	Locked both scripts in production
Cheating flags	Member 4	Similarity flag list, review tool	Violations, snapshots, host dashboard
Question side panel	Member 5	Filter API	Panel design and topic and difficulty filters

Joint function	With	Member 2 built	Other member built
Mistake book and AI hints	Members 1, 5	Failing verdict data	Hook in PlayController.java, My mistakes, AI hints
Library of books	Member 5	Book management and licence review	Library page and licence display

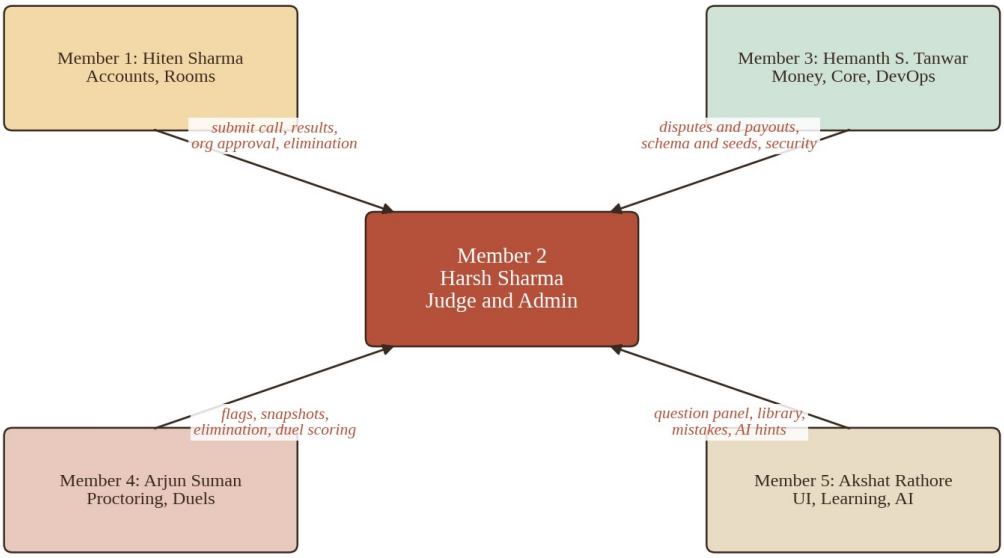


Figure 6.1: Collaboration map of Member 2 with the other four members

6.2 Collaboration with Member 1 (Hiten Sharma)

Member 1 owns accounts, one-time passwords, rooms and matches. Member 2's work meets Member 1's work in four places.

A. Submission and verdict

This is the most important joint function, because it connects a player's code to a stored result. Member 2 built the judge side in the second week: the code submit function with language and question id (13 July), the run of every test case (14 July) and the stored verdict, time and memory (15 July). Member 1 built the other side two weeks later: the submission endpoint in `PlayController.java` (29 July) and, the next day, the judge call and the server-side on-time check (30 July). On 31 July Member 1 tested a full solo room with two users while Member 2 was finishing the known good and known bad solutions and the stored code. The two sides worked from the interface the two members had agreed: the endpoint passes the room, question, language and code, and the judge returns a verdict and a time.

The file `PlayController.java` is listed in the main report among the files of Module 3, and the work log shows that the submission endpoint inside it was written by Member 1. It is therefore a shared file, and this report counts it as a joint function and not as work of Member 2 alone.

B. Results and elimination

Member 1 built final ranks and the tie-break rule on 3 August and the results page data on 4 August. The score that feeds the ranking comes from the verdicts of Member 2's judge together with the time bonus. On 17 August Member 1 built the host elimination of a player with a reason, and on 18 August blocked rejoin and removed the eliminated score from the ranking. Member 2's review tool for eliminated players (27 August) reads exactly these records, so an administrator can check that every elimination had a reason.

C. Confirm results and disputes

Member 2 built the dispute feature in Week 7 (17 to 21 August). Member 1 built the host confirm-results button on 24 August and the 24-hour auto-confirm when there is no dispute on 25 August, and on 26 August sent the final rank list to the money module. The dispute is therefore the guard between the confirmed result and the payout: a result is confirmed after 24 hours only if no dispute exists, and an open dispute stops the money step. The two members tested result locking together with Member 3 on 27 August.

D. Organisation approval and the admin gate

On 31 August Member 1 built the organisation host application form, on 1 September the admin approval flow for organisations, and on 2 September the higher limits for approved organisations. The approval flow is an administrator action, so it sits behind the admin gate that Member 2 built on 3 August and uses the accounts list that Member 2 built on 4 August. In the other direction, the admin gate depends on the signed session token created by `SecurityConfig.java, JwtService.java`, which Member 1 audited in the first week (6 to 8 July).

E. End-to-end checks

Member 1 wrote a script that signs up, creates a room and joins it (7 September), extended it to solve a question and show results (8 September) and ran it (9 September). Because the script solves a question, it exercises the judge of Member 2 in every run. Member 2 ran the same path by hand as a stranger would on 14 September. Member 1 re-ran the script on 30 September and Member 2 re-ran the judge checks on 1 October.

6.3 Collaboration with Member 3 (Hemanth Singh Tanwar)

Member 3 owns money, the platform core and DevOps. The main report names Member 3 as the partner for the admin and moderation interface, because disputes and payouts cross the line between the two modules.

A. Disputes block settlement

Member 2 made a dispute block the payout on 18 August. Member 3 built the money side in the surrounding weeks: the entry fee checkout (17 August), refunds (19 and 20 August), the payout details (24 August), the payout job with the 1% fee (26 August) and the prize split by rank (27 August). The main report states the rule that joins the two: an open dispute blocks settlement. The money module reads the dispute record that Member 2's module writes. The administrator money tab shows the totals and the ledger that Member 3 maintains; the administrator money totals endpoint was one of the smoke test checks of the main report.

B. Failed payouts and retry

Payout is a job that can fail. On 24 August Member 3 built the payout details with a one-time password to the profile mobile, on 25 August added the Razorpay account validation and on 26 August built the payout job. During the same days Member 2 built the failed payouts screen (24 August) and the retry button (25 August). The retry button asks for the payout to be attempted again; it does not contain money logic. This is a clean division: Member 3 decides how money moves, Member 2 gives the administrator a way to see when it did not and to ask for another attempt. On 28 August Member 2 tested the admin tools and Member 3 added the ledger balance check, so the screens and the ledger were tested in the same week.

C. Security of the admin scripts

Member 2 protected `the make-admin script and removed the fix-admin script` from deployment on 7 August. Member 3 locked both scripts in production on 4 September, after the security tests of 31 August to 3 September. The two actions cover the same risk in the code (Member 2) and in the deployment (Member 3).

D. Schema, seed data and security tests

The question and submission tables are part of the schema that Member 3 owns. Member 3 fixed the schema V1 tables on 13 July and finished the migration runner on 14 July. Member 3 also removed fake data from the seed files on 9 July, which allowed Member 2 to load only real starter questions on 22 and 23 July and to confirm on 18 September that the seed files contain only reference data. In the security tests of 31 August to 3 September Member 3 checked that every route, including the routes of Member 2, uses parameterised queries, tested SQL injection and session fixation, and fixed the findings.

E. Rate limits, load and production

Member 3 added rate limits on login and one-time passwords (20 July) and input validation (21 July). Member 2 added the concurrency cap and the one-pending rule for the judge a week later (27 and 28 July). The two sets of limits protect different doors of the same server. In September Member 3 simulated 200 and then 500 players polling one room (7 and 8 September), while Member 2 loaded the judge with 200 queued submissions (7 and 8 September) and tuned its concurrency (9 September). Member 3 deployed Milestone 1 to staging on 30 July, deployed to production on 22 September and checked every payout after the first tournament on 24 September; Member 2 reviewed the disputes from the same event on the same day and closed the open ones on 29 September.

6.4 Collaboration with Member 4 (Arjun Suman)

Member 4 owns proctoring, the host dashboard, duels and teams. The main report names Member 2 as the partner for the interface of the duels and teams module with the judge.

A. Cheating flags and the host dashboard

Proctoring produces evidence of one kind: violations and snapshots. The similarity check of Member 2 produces evidence of another kind. They meet on the host dashboard and in the admin screens. Member 4 built the host dashboard live player list (17 August), the camera status and violation counts (18 August), the snapshot viewer (19 August) and the eliminate button (20 August). Member 2 had produced the similarity flag list for the host dashboard on 12 August and shown it in the admin screen on 13 August, and then built the review tool for

flagged cheating on 26 August. The review tool therefore gives an administrator one place to see both kinds of evidence. Because Member 4 added a job that deletes snapshots after 30 days (21 August), a review that depends on snapshots should be finished inside that period.

B. False locks and fairness

From 24 to 28 August Member 4 ran a 10-person proctoring test, counted false locks and tuned the grace time. Member 2 tuned the similarity threshold in the same spirit on 14 August, so that the platform raises a flag only when it is worth a human look. Both members therefore treated false alarms as a design problem and not as an afterthought. On 29 September Member 4 tuned the lock rules after reviewing the logs of the first event; Member 2 reviewed the remaining flagged cheating the day before.

C. Duels, teams and scoring

Duels reuse the same room and scoring code, and every duel or team question is judged by the same judge. Member 4 made the team score count each question once (3 August), built automatic duel pairing (4 August) and tested duo, squad and duel play (7 August). The pass or fail result that these features rely on comes from Member 2's verdict. The duel picker draws on the same question bank that Member 2 manages, so a question added in the admin screen is available for duels at once; the back-office page says so on its question bank tab.

D. The first event

During the first event, Member 4 monitored locks on 22 September and snapshots on 23 September, and fixed urgent proctoring problems on 24 September. Member 2 reviewed flagged cheaters on 23 September and the disputes of the event on 24 September. The same days thus show a chain of work: proctoring raises a flag, an administrator reviews it, and a dispute, if any, is decided.

6.5 Collaboration with Member 5 (Akshat Rathore)

Member 5 owns the design system, learning features and the AI coach. The main report names Member 2 as the partner for the interface of the learning module, because the learning data begins with a judge verdict.

A. The question side panel

Member 5 designed the create-room page with the side question panel on 20 July and the topic and difficulty filters on 21 July. The panel needs questions that can be filtered by topic and difficulty. Member 2 built the filter API on 24 July, after the starter questions had been loaded on 22 and 23 July. The panel and its data therefore came together in the same week, and Member 5 tested the page at phone width on 24 July.

B. Mistakes and the failing case

Member 5 agreed a hook in `PlayController.java` with Member 1 for saving wrong submissions on 3 August and saved wrong submissions automatically on 4 August. On 5 August the My mistakes list was built, showing the last attempt, and on 6 August it was extended to show the failing case and a retry button. The failing case is the output of Member 2's judge. The mistakes table is listed among the tables of Module 3 in the main report, so the table is shared by design: Member 2's judge writes the failure, and Member 5's learning page reads and presents it.

C. AI hints after a failed attempt

Member 5 wired AI hints through `AiController.java` on 21 August and built the AI hint shown after a failed attempt on 31 August. A hint is only offered after a failed verdict, so the trigger of the feature is the judge's result. On 1 and 2 September Member 5 added the daily hint limit and a friendly message when the limit is reached. The hints support learning, because the coach gives hints before answers.

D. The library of books

The library has two sides. Member 2 built the book management screen on 6 August, where an administrator checks the licence and the approval status of a book. Member 5 built the free book library page grouped by subject on 13 August and showed the licence on each book on 14 August. A book appears to players only after the administrator's review, so the two screens together enforce the licence rule.

E. Look and feel of the admin pages

The administrator page uses the shared components and tokens of the design system that Member 5 built in July, so the back office looks like the rest of the product. Member 5 ran the design consistency pass on 3 and 4 September, the contrast check on 7 and 8 September, and added focus states on 9 September, all of which apply to the admin page as well. After the first event, Member 5 collected feedback from players (24 September) and reviewed it (28 September); Member 2 used the same period to add questions to the bank (30 September).

6.6 Team-Wide Activities

Some activities involved all five members. In the first week, all members tested or audited the existing code, and on 10 July each wrote the decisions that the others would build on; Member 2's decision record was the note about the judge service and its limits. Three milestones were shared: the first playable match on 2 August, the full paid test tournament on 30 August and the first deployed release on 27 September. Member 1 recorded the code freeze on 21 September and Member 3 the final pre-deploy checks, while Member 2 prepared the handling of user reports for the launch. After the first event, Member 2 attended the team retro on 25 September. In the last week Member 2 reviewed the pull requests of teammates on 2 October, and all members did the final review and submission preparation on 6 October.

The main report records that shared files such as the schema, the server entry point and the API helper were edited only by Member 3 and Member 5, and that other members asked for changes by pull request. Member 2 followed the same rule for the shared files that touch the judge and the admin tools.

6.7 Summary of Collaboration

Member 2 worked with all four other members. With Member 1 the shared object was the submission and its result; with Member 3 it was the money that moves after a dispute is settled or a payout fails; with Member 4 it was the evidence of cheating; and with Member 5 it was the learning material that comes out of a failed attempt and the questions and books that go in. In every case the interface was small and clear, and the work of the two sides was done in the same week. This pattern is what allowed ten modules built by five people to run together at the first real tournament.

7. TESTING AND VERIFICATION

7.1 Test Strategy

Member 2 used four kinds of testing. **Reference solutions** (known good and known bad programs in each language) checked that the judge accepts what it should accept and rejects what it should reject. **Feature tests** were done at the end of every build, so that a feature was tested on the day or in the week it was finished. **Stress tests** sent a large number of submissions and hostile inputs to the judge. **Stranger tests** walked through the application as a new user would, to find problems that the builders no longer see.

7.2 Tests Recorded in the Work Log

Table 7.1: Tests carried out by Member 2

Date	Test	Outcome or follow-up
06 to 07-07	Judge with Java, Python and C++ programs	Behaviour recorded for each language
08-07	Time and memory limits	Limits confirmed
17-07	Question update API	Tested after it was built
29 to 30-07	Known good and known bad solutions	Verdicts checked for every language
11-08	Similarity on sample submissions	Threshold tuned on 14-08
21-08	Dispute flow	Raise, block, decide and close tested together
28-08	Admin tools	Dispute, payout, flag and elimination tools tested
08-09	Judge load test with 200 queued submissions	Results noted; concurrency tuned on 09-09
10-09	Huge inputs and infinite loops	Runs stopped at the limits
11-09	All limits enforced	Confirmed
14-09	App tested as a stranger from sign-up to match	Bugs logged in the tracker on 15-09
01-10	Judge checks re-run for every language	No verdict changed after the launch

7.3 Results from the Main Report's Smoke Test

The automated smoke test described in the main report exercises several of Member 2's features. The cases that concern the judge, the question bank and the admin gate all passed.

Table 7.2: Smoke test cases that concern Member 2

No.	Test case	Result	Module
3	Bootstrap data hides hidden test cases (19 questions, 7 rooms)	Pass	Module 3
4	Administrator gate admits an admin and lists users	Pass	Module 4
8	Correct JavaScript passes 4 of 4 hidden cases and scores +140	Pass	Module 3

No.	Test case	Result	Module
9	Wrong answer fails and is booked in the mistake book	Pass	Module 3
10	Non-JavaScript verdict is flagged as simulated	Pass	Module 3
13	Administrator money totals endpoint	Pass	Module 4

A run with a non-administrator account correctly failed the administrator checks, which confirms that the gate works for the right reason. The checks that failed in the administrator run did not concern Member 2: two were the AI-coach checks that need a key, and two were proctoring checks that failed because the script created a room without proctoring.

7.4 Problems Found and Fixed

Testing led to a steady flow of fixes. During the build Member 2 fixed bugs found in the similarity tool (14 August) and the dispute flow (21 August). After the stranger test of 14 September, the problems were logged in the tracker on 15 September, judge bugs were fixed on 16 September and admin bugs on 17 September. The week of 14 to 18 September was a bug bash in which the whole team tested the product from the outside.

7.5 Known Issues That Remain

- Only JavaScript is really executed; the other four languages are simulated and flagged as such.
- The in-process sandbox is not a hardened security boundary, so the judge must not be exposed to untrusted users in its present form.
- Similarity is a threshold-based hint and needs human review; it can flag honest players who write similar short answers.
- Browser proctoring cannot detect a second device, so flags of both kinds are evidence for review and not proof.

8. WEEK-BY-WEEK SUMMARY

This chapter follows the project plan of twelve weeks plus a final period, as set out in the main report, and records what Member 2 did in each. Every full week contains five working days and 14.5 logged hours. The final period, from 28 September to 6 October, contains seven days and 20 hours.

Table 8.1: Weekly summary of Member 2's work

Week	Dates	Focus	Hours
1	6 to 10 Jul	Judge audit: Java, Python, C++, limits, verdict map, judge service note	14.5
2	13 to 17 Jul	Code submit, all test cases, stored verdict, question create, read and update API	14.5
3	20 to 24 Jul	Admin screens for questions and test cases, starter questions, filter API	14.5
4	27 to 31 Jul	Concurrency cap, one pending submission, good and bad solutions, stored code	14.5

Week	Dates	Focus	Hours
5	3 to 7 Aug	Admin login, users list, question and book management, admin script protection	14.5
6	10 to 14 Aug	Similarity comparison, flag list in host dashboard and admin, threshold tuning	14.5
7	17 to 21 Aug	Dispute form, payout block, review screen, decide and close, test	14.5
8	24 to 28 Aug	Failed payouts screen and retry, review tools for cheating and elimination, test	14.5
9	31 Aug to 4 Sep	Final dashboard, dispute and payout panels, flagged cheating panel, medium and hard questions	14.5
10	7 to 11 Sep	Load test with 200 submissions, concurrency tuning, hostile input tests, limits check	14.5
11	14 to 18 Sep	Stranger test, bug tracker, judge and admin bug fixes, seed file check	14.5
12	21 to 25 Sep	Report handling, report review, flagged cheaters, first event disputes, retro	14.5
Final	28 Sep to 6 Oct	Remaining reports, disputes closed, more questions, judge re-check, admin guide, review	20.0
Total			194.0

8.1 Weeks 1 and 2: Audit and Judge Core

Week 1 was spent reading and testing. The judge was tried with Java, Python and C++ programs, its time and memory limits were confirmed, the verdict mapping was written down, and the judge service and its limits were recorded as a decision for the team. **Week 2** built the first working path: submit code with a language and question id, run every case, store the verdict, and manage questions through a create, read and update API. This week was the base for everything that followed. In the same week Member 3 finished the repository layer and Member 5 built the first components, so the three kinds of foundation, data, judge and interface, were laid together.

8.2 Weeks 3 and 4: Question Bank and Judge Hardening

Week 3 gave the question bank its screens and its first content: admin screens for questions and test cases, starter questions for six topics, and the filter API for the side panel that Member 5 designed. **Week 4** made the judge dependable: a concurrency cap, one pending submission per player per question, known good and known bad solutions in every language, and stored code for later comparison. Member 1 built the submission endpoint in the same week, and the first playable match was ready for Milestone 1 on 2 August.

8.3 Weeks 5 and 6: Admin Back Office and Similarity

Week 5 protected the back office and built its first management screens, namely admin login protection, the users list with block and unblock, the question management screen and the book management screen. The make-admin script was protected and the fix-admin script was

removed from deployment. **Week 6** added an integrity tool: a similarity comparison of submissions in a room, tested on samples, a flag list for the host dashboard and the admin screen, and a tuned threshold. Member 4 built the host dashboard that would show the flag list in the following week.

8.4 Weeks 7 and 8: Disputes, Payouts and Review Tools

Week 7 built the dispute feature from form to closing action, with the rule that a dispute blocks payout. **Week 8** added the failed payouts screen and the retry button, and the review tools for flagged cheating and eliminated players, and ended with a test of all admin tools. These two weeks were the busiest in collaboration: Member 1 built the elimination logic, host confirm and auto-confirm, and Member 3 built the payout job and the settlement split. Milestone 2, the full paid test tournament, followed on 30 August.

8.5 Weeks 9 and 10: Dashboard, Questions and Load

Week 9 assembled the final admin dashboard with its panels for disputes, payout failures and flagged cheating, and added medium and hard questions for every topic. **Week 10** was a stress week. A script sent 200 queued submissions, the concurrency was tuned, huge inputs and infinite loops were tested and the enforcement of every limit was confirmed. Member 3 was running load simulations of 200 and 500 players on the same days.

8.6 Weeks 11 and 12: Bug Bash and Launch

Week 11 tested the application as a stranger would, from sign-up to a match, logged the bugs in the tracker, fixed the judge and admin bugs, and confirmed that the seed files hold only reference data. **Week 12** was the launch week. User report handling was set up and used, flagged cheaters and the disputes of the first event were reviewed, and the team retro was held on 25 September. Milestone 3, the first deployed release, was on 27 September.

8.7 Final Period: Closing the Loop

In the final period Member 2 reviewed the remaining user reports and flagged cheating (28 September), closed the open disputes (29 September), added more questions (30 September), re-ran the judge checks for every language (1 October), reviewed the pull requests of teammates (2 October), wrote the short admin guide (5 October) and completed the final review and submission preparation (6 October).

9. CHALLENGES, DECISIONS AND LIMITATIONS

9.1 Challenges and How They Were Met

Table 9.1: Challenges and responses

Challenge	Response of Member 2
Five languages but only one real executor	Tested all five in the audit, recorded the limits, flagged simulated verdicts in the response, kept all judging in one module for later replacement
Bursts of submissions could overload the judge	Concurrency cap, one pending submission per player per question, and a measured load test with 200 queued runs

Challenge	Response of Member 2
Hostile or broken programs	Time limit, tests with huge inputs and infinite loops, confirmation that all limits are enforced
Hidden test cases must stay secret	Bootstrap data strips hidden cases; verified by smoke test case 3
Cheating cannot be proved by software alone	Similarity and proctoring flags go to human review tools with a tuned threshold
Money must not move during a disagreement	Dispute record blocks payout until an administrator decides
Dangerous helper scripts	The make-admin script protected, the fix-admin script removed from deployment; Member 3 locked both in production
Dependence on four other members	Small interfaces agreed in advance; the two sides of each joint function built in the same week

9.2 Delivered, Simulated and Planned

The main report insists on separating what really works from what is simulated or planned. The same separation is applied here to the scope of Member 2.

Table 9.2: Status of Member 2's features

Feature	Status in this build
JavaScript judging with hidden cases	Delivered (real execution in a restricted sandbox context)
Java, Python, C++ and C judging	Simulated and flagged in the response; Judge0 is future scope
Question bank, admin screens, filters, starter questions	Delivered
Admin gate, users list, block and unblock, book licence review	Delivered
Dispute raise, block, review and close	Delivered
Failed payouts screen and retry	Delivered; payouts use the simulated gateway of the main report in the tested build
Similarity comparison and flag list	Delivered according to the work log
Cheating and elimination review tools, dashboard	Delivered

9.3 Notes on the Sources

The work log and the main report were written for different purposes, and some of their details do not line up exactly. These points are recorded so that the reader can judge them.

- The log says that the verdict, time and memory of each run were stored (15 July). The main report lists a `time_ms` column in the submissions table but does not name a memory column. This report follows the log for the activity and the report for the schema.

- The log mentions that the judge service and its limits were written down in Week 1. The main report states that the judge is simulated for four languages and that Judge0 is future scope; this report follows the main report on what is delivered.
- The similarity check, the concurrency cap and the payout retry button appear in the log but are not described in the main report. They are reported here as logged.
- The log of Member 3 refers to a Razorpay test account and live-mode checklist, while the main report calls the payment gateway simulated in the tested build. The retry feature of Member 2 is therefore described without claiming that real money was moved.
- The main report lists `PlayController.java` under Module 3; the log of Member 1 shows that the submission endpoint inside it was written by Member 1. This report treats the file as shared.

9.4 Limitations

- The judge is not a security sandbox; it must be replaced by an isolated service before it is exposed to untrusted users.
- Languages other than JavaScript are judged by simulation.
- The similarity check is a screening tool; it cannot prove copying and does not detect help from another person who writes a different solution.
- The review tools depend on the data of other modules, for example snapshots that are deleted after 30 days.

10. CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

Member 2 delivered the two modules that make a HackAgon contest fair and manageable. The code judge turns every submission into a stored verdict, hides the test cases from the browser, and is protected by limits that were tested under load. The question bank gives hosts a supply of questions with filters for topic and difficulty. The back office lets administrators control content and accounts, review similarity and cheating flags, decide disputes that block payouts, and retry failed payouts, and it was used with real data at the first tournament. The work was done in 67 working days and 194 logged hours, and every logged day is marked completed.

Equally important, the work was done in collaboration. The submission path was built with Member 1, the dispute, payout and security functions with Member 3, the cheating evidence with Member 4, and the question panel, mistake data and library with Member 5. Each joint function has two halves that were built in the same week, and the project ran as one system because those halves fit.

10.2 Lessons Learnt

- **Audit before building.** A week spent testing the existing judge made every later decision faster and honest.
- **Reference solutions are cheap insurance.** Known good and known bad programs allowed quick re-checks after every change, including the re-check after launch.

- **Limits belong together.** The time limit, the concurrency cap and the one-pending rule only protected the server when they were tested as a set.
- **Make the human the final judge.** Similarity and proctoring flags are hints; the review tools let a person decide.
- **Agree small interfaces early.** Because the interfaces with other members were small and agreed in advance, the two sides of each function could be built in parallel.

10.3 Future Scope

- Replace the in-process judge by Judge0 or a similar isolated service for real execution in Java, Python, C++ and C.
- Enforce the age rule for paid contests and extend the admin tools with identity checks for large prize pools, in line with the main report.
- Move snapshots to object storage so that reviews do not depend on temporary disk or on a 30-day deletion window.
- Improve the similarity measure and show the compared code side by side in the review tool.
- Add more questions in every topic and a review workflow for questions suggested by hosts.

REFERENCES

[1] Oracle. Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/

[2] VMware. Spring Boot 3.2 Reference Documentation. https://docs.spring.io/spring-boot/docs/3.2.0/reference/html/

[3] Oracle Corporation. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc

[4] PingCAP. TiDB Cloud Documentation. https://docs.pingcap.com/tidbcloud

[5] Jones, M., Bradley, J., Sakimura, N. JSON Web Token (JWT), RFC 7519, IETF, 2015.

[6] OWASP Foundation. OWASP Top Ten and Authentication Cheat Sheet. https://owasp.org

[7] Judge0. Judge0 Open-Source Online Code Execution System. https://judge0.com

[8] Render. Render Documentation: Web Services. https://render.com/docs

[9] HackAgon Project Report, Arya College of Engineering and IT, Jaipur, Session 2026-27.

[10] HackAgon Daily Work Log, 6 July 2026 to 6 October 2026, Members 1 to 5.

APPENDIX A: COMPLETE DAILY WORK LOG OF MEMBER 2

The table below reproduces the daily work log of Member 2 (Judge, Admin) for 6 July 2026 to 6 October 2026. It contains 67 entries and 194 hours. Every entry is marked Completed in the source log.

Table A.1: Daily work log of Member 2

Date	Hrs	Work description
06-07-2026	3	Tested JudgeService.java with a sample Java program
07-07-2026	2.5	Tested judge with Python and C++ programs
08-07-2026	4	Confirmed time and memory limits
09-07-2026	3	Noted how each verdict maps
10-07-2026	2	Wrote down which judge service is used and its limits
13-07-2026	3	Built code submit with language and question id
14-07-2026	2.5	Ran every test case in the judge
15-07-2026	4	Stored verdict, time and memory
16-07-2026	3	Built question create and read API
17-07-2026	2	Built question update API and tested it
20-07-2026	3	Built admin screen to add questions
21-07-2026	2.5	Built admin screen to add test cases
22-07-2026	4	Loaded starter questions for arrays and strings
23-07-2026	3	Loaded starter questions for linked lists, stacks, queues, trees
24-07-2026	2	Built filter API for question side panel
27-07-2026	3	Added limit on concurrent judge runs
28-07-2026	2.5	Allowed one pending submission per player per question
29-07-2026	4	Wrote known good solutions for each language
30-07-2026	3	Wrote known bad solutions and checked verdicts
31-07-2026	2	Stored submitted code for similarity checks
03-08-2026	3	Added admin login protection
04-08-2026	2.5	Built users list with block and unblock
05-08-2026	4	Built question management screen
06-08-2026	3	Built book management screen
07-08-2026	2	Protected the make-admin script and removed the fix-admin script from deployment
10-08-2026	3	Wrote similarity comparison for submissions in a room
11-08-2026	2.5	Tested similarity on sample submissions
12-08-2026	4	Produced flag list for the host dashboard
13-08-2026	3	Showed flag list in admin
14-08-2026	2	Fixed bugs and tuned threshold

Date	Hrs	Work description
17-08-2026	3	Built player dispute form
18-08-2026	2.5	Made dispute block payout
19-08-2026	4	Built admin dispute review screen
20-08-2026	3	Added decide and close actions
21-08-2026	2	Tested dispute flow
24-08-2026	3	Built screen for failed payouts
25-08-2026	2.5	Added retry button for payouts
26-08-2026	4	Built review tool for flagged cheating
27-08-2026	3	Built review tool for eliminated players
28-08-2026	2	Tested admin tools
31-08-2026	3	Built final admin dashboard layout
01-09-2026	2.5	Added disputes and payout failures panels
02-09-2026	4	Added flagged cheating panel
03-09-2026	3	Added medium questions for every topic
04-09-2026	2	Added hard questions for every topic
07-09-2026	3	Prepared script to send 200 queued submissions
08-09-2026	2.5	Ran judge load test and noted results
09-09-2026	4	Tuned concurrency
10-09-2026	3	Tested huge inputs and infinite loops
11-09-2026	2	Confirmed all limits are enforced
14-09-2026	3	Tested the app as a stranger would, signup to match
15-09-2026	2.5	Logged bugs found in the tracker
16-09-2026	4	Fixed judge bugs
17-09-2026	3	Fixed admin bugs
18-09-2026	2	Confirmed seed files contain only reference data
21-09-2026	3	Set up handling of user reports
22-09-2026	2.5	Reviewed user reports
23-09-2026	4	Reviewed flagged cheaters
24-09-2026	3	Reviewed disputes from the first event
25-09-2026	2	Held team retro
28-09-2026	3	Reviewed remaining user reports and flagged cheating
29-09-2026	2.5	Closed open disputes
30-09-2026	4	Added more questions to the question bank
01-10-2026	3	Re-ran judge checks for every language
02-10-2026	2	Reviewed teammates' pull requests

Date	Hrs	Work description
05-10-2026	3	Wrote short admin guide
06-10-2026	2.5	Final review and submission preparation

APPENDIX B: DATED JOINT WORK WITH OTHER MEMBERS

This appendix lists the entries of Member 2 next to the matching entries in the logs of the other members (M1 to M5), as an aid to Chapter 6.

Table B.1: Dated joint work

Date	Member 2	Other members	Function
10-07-2026	Wrote down the judge service and its limits	M1 reviewed data model with M3; M4 wrote proctor API contract; M5 wrote style guide	Week 1 decisions
13 to 15-07-2026	Code submit, all test cases, stored verdict	M3 fixed schema V1, finished migration runner and repo layer	Judge and data layer
22 to 23-07-2026	Loaded starter questions	M3 had removed fake data from seed files (09-07)	Seed data
24-07-2026	Built filter API for question side panel	M5 designed side panel and filters (20 to 21-07)	Question side panel
27 to 28-07-2026	Concurrency cap; one pending submission	M3 rate limits and validation (20 to 21-07)	Server protection
29 to 31-07-2026	Known good and bad solutions; stored code	M1 submission endpoint (29-07), judge call and on-time check (30-07), solo room test (31-07)	Submission and verdict
07-08-2026	Protected the make-admin script, removed the fix-admin script	M3 locked both scripts in production (04-09)	Admin script security
06-08-2026	Built book management screen	M5 built library page (13-08) and licence display (14-08)	Library of books
12 to 13-08-2026	Similarity flag list in host dashboard and admin	M4 host dashboard, violation counts, snapshot viewer (17 to 19-08)	Cheating flags
17 to 21-08-2026	Dispute form, payout block, review, decide, close	M1 elimination (17 to 18-08); M3 refunds and entry fee (17 to 20-08)	Disputes
24 to 25-08-2026	Failed payouts screen and retry button	M3 payout details, Razorpay validation, payout job (24 to 26-08)	Failed payouts
24 to 25-08-2026	(Dispute guard on results)	M1 host confirm results and 24 h auto-confirm	Results and disputes
26 to 27-08-2026	Review tools for flagged cheating and eliminated players	M4 eliminate button and false lock counts; M1 elimination logic	Elimination review
27-08-2026	Joint test of result locking	M1 and M3 tested result locking	Results, money
07 to 09-09-2026	Judge load test; tuned concurrency	M3 load simulation up to 500 players (07 to 08-09)	Load
14-09-2026	Tested the app as a stranger	M1 end-to-end script (07 to 09-09); M3 production set-up	End-to-end

Date	Member 2	Other members	Function
23 to 24-09-2026	Reviewed flagged cheaters and first event disputes	M4 monitored locks and snapshots; M3 checked every payout	First event
25-09-2026	Team retro	All members held the retro	Team
29-09-2026	Closed open disputes	M3 reviewed Razorpay webhook logs; M4 tuned lock rules	First event follow-up
01-10-2026	Re-ran judge checks for every language	M1 re-ran end-to-end script (30-09)	Regression